Vision-Only Landing of a Quadcopter on a Moving Ground Vehicle: Attitude-Compensated Estimation and a Closed-Loop Simulation Study in ROS 2

Dimuthu Wickramanayake
Email: dimuthu.billa@gmail.com

Abstract—We study the landing of a quadcopter on a moving ground vehicle using only a downward-facing monocular camera and a landing pad carrying three ArUco markers of different sizes, with no communication link between the aircraft and the vehicle. The system is implemented as a set of ROS 2 nodes: a marker detector that fuses all visible markers into one pad pose, a relative-state estimator, and a landing state machine that ends with a motor cut and a short free fall onto the pad. Because a multirotor must tilt to accelerate, the pose measured in the camera frame is rotated by the aircraft attitude into a gravity-aligned frame before it is used, and the pad's position and velocity are tracked with a constant-velocity Kalman filter. We evaluate three estimator variants on identical rendered camera images in a closed-loop simulator with wind, camera latency and flight-controller state errors. In a hover test with the pad passing at 1–9 m/s, the filtered estimator's speed RMSE is 0.07–0.14 m/s, against 0.48–0.98 m/s for unfiltered variants. Over 22 092 in-flight detections, the median position error of the camera-frame estimate grows from 2.6 cm below 2° of tilt to 1.62 m at 20°–30°, while the attitude-compensated estimates stay between 1.0 and 3.2 cm. In 28 closed-loop trials per variant across seven vehicle motions, the camera-frame variant never landed, the attitude-compensated variant landed 6 times and the filtered variant 7 times, at a mean distance of 6.3 cm from the pad centre. No landing succeeded for weaving, stop-and-go or 4.5 m/s vehicle motion. The code, simulator and data are open source.

Index Terms—quadcopter, autonomous landing, moving platform, fiducial markers, ArUco, Kalman filter, ROS 2

I. Introduction

Multirotor aircraft can take off and land vertically and hover in place, which makes them useful for inspection, delivery and search tasks. Their short flight time makes it attractive to launch and recover them from a ground vehicle, but landing on a vehicle that is moving is considerably harder than landing on the ground. The aircraft must find the landing pad, estimate where it is and how fast it is moving, match that motion despite wind, and touch down on a small area.

Many systems solve part of this problem by sharing state between the aircraft and the vehicle, for example by sending the vehicle's GNSS position or odometry over a radio link. Such a link cannot always be relied on. This paper therefore considers the case in which the aircraft carries everything it needs: one downward-facing camera, the attitude and velocity estimate of its own flight controller, and a known pattern of fiducial markers on the vehicle.

The work builds on an earlier prototype by the author and colleagues [1], which used a Raspberry Pi for vision, an Arduino Nano for flight control and a landing pad with three ArUco markers of different sizes, and which measured the speed of a moving target from a hovering aircraft. Closing the loop—flying the aircraft from the vision estimate all the way to touchdown—was left as future work. This paper addresses that step.

The specific contributions are:

1) A complete vision-only landing pipeline in ROS 2 (Jazzy): multi-marker pad detection, attitude-compensated relative-state estimation with a constant-velocity Kalman filter, and a landing state machine with a predicted-impact check before the motor cut.
2) A closed-loop simulator that renders the downward camera image so that the actual OpenCV ArUco detector runs in the loop, together with quadrotor and vehicle dynamics, gusting wind, camera latency and flight-controller state errors. It runs without Gazebo or a GPU.
3) A controlled comparison of three estimator variants on identical images, quantifying how estimation error depends on aircraft tilt and how each variant performs in closed-loop landing across seven vehicle motions.
4) A measurement of how the three-marker pad's detection rate varies with height.

All software, the simulator, the experiment scripts and the raw results are available at https://github.com/deBilla/moving-target-landing-ros2.

II. Related Work

Surveys of autonomous UAV landing are given by Gautam et al. [2], who compare GPS-based and vision-based methods and the control approaches used with them, by Kong et al. [3], who classify vision-based landing

systems, and more recently by Xin et al. [4], who separate static, dynamic (vehicle- and ship-based) and complex landing scenarios. We organise the work most relevant here into five groups.

A. Vision-guided landing on moving targets

Saripalli et al. [5] demonstrated real-time vision-guided landing of an autonomous helicopter on a static target, and Saripalli and Sukhatme [6] extended it to a moving target, recognising the target with Hu's moment invariants and tracking it with a Kalman filter. Wenzel et al. [7] showed a miniature UAV taking off from, tracking and landing on a moving carrier vehicle. Herissé et al. [8] used optical flow together with an IMU in a nonlinear controller that stabilises a VTOL aircraft relative to a moving textured platform, with a stability proof and quadrotor experiments. Lee et al. [9] applied image-based visual servoing [10], producing velocity references from image-space errors for an adaptive sliding-mode controller. Lee, Jung and Shim [11] detected a coloured marker on a vehicle on board in real time and landed on the moving vehicle in outdoor flights. Falanga et al. [12] landed a quadrotor on a moving platform using only onboard sensing and computation, combining platform detection, motion estimation and trajectory planning on low-cost hardware. Ghommam and Saad [13] used a single downward camera for relative position and divided the landing into search, homing and landing phases with adaptive backstepping control. Gautam et al. [14] used pure-pursuit guidance with a closing-velocity controller and landed on a slowly moving target in 22 of 27 field trials.

B. Relative-state estimation

Many of these systems estimate the platform's relative position and velocity with a filter [15]. Borowczyk et al. [16] used a Kalman filter on relative position and velocity and landed a commercial quadcopter repeatedly on the roof of a car travelling at high speed. Araar et al. [17] designed a landing pad together with a relative pose estimator for it, and fused the visual pose with inertial measurements to obtain a high-rate estimate. Paris et al. [18] used an extended Kalman filter that switches from (simulated) GNSS at long range to fiducial markers at short range. Feng et al. [19] localised a mobile platform with a Kalman filter and a gimbaled camera. A gimbal, or fusion with inertial data, is one way to account for the fact that a body-fixed camera rotates with the aircraft. In this paper we quantify the size of that effect on a fixed downward camera during pursuit, and its consequence for closed-loop landing.

C. Trajectory planning and control

Several works plan the approach explicitly. Vlantis et al. [20] used nonlinear model predictive control (MPC) with a forward-looking camera to land on an inclined platform carried by a mobile robot, keeping the platform in the field of view as a constraint. Feng et al. [19] and Guo et al. [21] used MPC guidance, the latter replanning with a variable sampling time as the aircraft nears the platform. Hu and Mishra [22] generated time-optimal landing trajectories onto a heaving and pitching platform. Paris et al. [18], [23] addressed fast landing in turbulent wind, first with receding-horizon planning and robust control and then with explicit wind estimation. The landing task of the Mohamed Bin Zayed International Robotics Challenge (MBZIRC) 2017 produced field-tested systems: Baca et al. [24] predicted the vehicle's motion with a nonlinear model and used MPC to land on a car at 15 km/h, and Beul et al. [25] combined camera-based perception with fast trajectory planning for the same task. Daly et al. [26] took a different approach in which the aircraft and a ground vehicle cooperate through a decentralised controller, which requires communication between them.

D. Learning-based landing

Polvara et al. [27] trained a hierarchy of deep Q-networks to land on a static marker from low-resolution downward images. Rodriguez-Ramos et al. [28] trained a DDPG agent in a Gazebo-based framework [29] for landing on a moving platform and transferred it to real flights. Ladosz et al. [30] learned landing on a moving platform end-to-end from camera images, including vertical velocity control, and reported better performance in simulation than with hand-extracted relative position and velocity features.

E. Fiducial markers and landing pads

Square fiducial markers such as AprilTag [31], [32] and ArUco [33], [34] give a full 6-DOF pose from a single image and are widely used for landing pads. Planar pose can be computed accurately with infinitesimal plane-based pose estimation (IPPE) [35]. A known problem for landing is that a single marker is either too small to detect from high up or too large to stay in the image near touchdown. Romero-Ramirez et al. [36] addressed this with fractal markers, square markers nested inside one another, and Khazetdinov et al. [37] proposed embedded ArUco markers, nested markers detectable with the standard ArUco detector, reporting a mean landing error of 2.03 cm in ROS/Gazebo simulation. Other ArUco-based landing systems include those of Wubben et al. [38], validated in simulation and real flights, and Lebedev et al. [39]. Our earlier prototype [1] used three separate ArUco markers of different sizes on one pad for a moving target, and measured target speed from a hovering aircraft.

F. Position of this work

Compared with the systems above, this work restricts the sensing to a body-fixed downward camera (no gimbal), the flight controller's own state estimate, and a passive

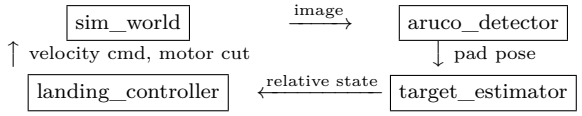


Fig. 1. ROS 2 node graph. sim_world also publishes the flight-controller state (attitude, velocity, position), which the estimator and controller subscribe to.

marker pad, with no communication with the vehicle. Rather than proposing a new controller, it isolates the estimation step: three estimator variants run on identical rendered images in closed loop, which lets us measure how much of the closed-loop performance each step accounts for. The software is built on ROS 2 [40] and OpenCV [41] and is released with its simulator and data.

III. System Overview

A. Frames and notation

The world frame $\mathcal{W}$ is gravity-aligned with z up. The body frame $\mathcal{B}$ is forward-left-up, and R_{WB} is the aircraft attitude reported by the flight controller. The camera frame $\mathcal{C}$ follows the OpenCV convention (x right, y down, z along the optical axis); the camera is rigidly mounted looking straight down, so

$$R_{BC} = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 0 & 0 \\ 0 & 0 & -1 \end{bmatrix}, \tag{1}$$

with the lens at $p_{BC} = (0, 0, -0.05)\,\mathrm{m}$ from the centre of gravity. The pad frame $\mathcal{P}$ has its origin at the pad centre and z normal to the pad.

B. Software architecture

The system consists of four ROS 2 nodes and a logger (Fig. 1). In simulation, sim_world publishes the camera image and the flight-controller state and accepts velocity commands and a motor-cut command; on an aircraft these interfaces correspond to the camera driver and to the offboard velocity interface of an autopilot such as PX4 [42]. aruco_detector publishes the pad pose, target_estimator publishes the relative state, and landing_controller closes the loop.

C. Landing pad

The pad is a 1 m square carrying three markers from the ArUco DICT_4X4_50 dictionary [33]: a large marker (32 cm), a medium marker (20 cm) and a small marker (16 cm) at the pad centre. The large and medium markers are placed off-centre around it. The small marker is centred because that is where the camera points in the final approach; at 0.3 m camera height a 70° lens sees only about $0.42\,\mathrm{m} \times 0.31\,\mathrm{m}$ of the pad. In early tests a 10 cm centre marker left the camera's view in the last 0.3 m unless the aircraft was within about 3 cm of the centre, so the small marker was enlarged to 16 cm.

D. Detection and pad pose

Markers are detected with OpenCV's ArUco module [33], [34] with sub-pixel corner refinement. Markers whose shortest side is below 14 pixels are discarded because their depth estimate is unreliable. The pose of the largest visible marker is solved with the IPPE method for squares [35] and shifted to the pad centre using the known layout. When more than one marker is visible, this pose initialises an iterative PnP solution over all visible corners of all pad markers. The result is the rotation R_{CP} (from the Rodrigues vector r) and the pad-centre position $t = (t_x, t_y, t_z)$ in the camera frame.

E. Relative-state estimation

We compare three estimators that receive the same detections.

Camera frame (CF). The pad position relative to the aircraft is $R_{BC}\,t$, treating the aircraft as level, and the altitude is t_z. Velocity is the finite difference of consecutive positions.

Attitude-compensated (AC). Each detection is converted to a world-frame pad position using the aircraft pose at the image timestamp:

$$p^W_{\mathrm{pad}} = p^W_{\mathrm{uav}} + R_{WB}\,(R_{BC}\,t + p_{BC})\,. \tag{2}$$

The aircraft pose is taken from a buffer of flight-controller states, matched to the image time to account for the processing latency. Pad velocity is the finite difference of consecutive world-frame positions, and the relative velocity is $v_{\mathrm{rel}} = v_{\mathrm{pad}} - v_{\mathrm{uav}}$.

Attitude-compensated with Kalman filter (AC+KF). The world-frame positions from (2) are the measurements of a constant-velocity Kalman filter [15] with state $x = [p^W_{\mathrm{pad}}, v^W_{\mathrm{pad}}]$ and white-acceleration process noise with standard deviation $2\,\mathrm{m/s^2}$ (the vertical diagonal terms are scaled by 0.1). The measurement standard deviation grows with range, $\sigma = 0.02 + 0.012\,\|t\|\,\mathrm{m}$, with 1.5σ on the vertical axis. Measurements whose squared Mahalanobis distance exceeds 16 are rejected. The filter is initialised by a least-squares line fit to at least three fixes spanning at least 0.1 s, and it is re-initialised if no measurement has been accepted for 1 s. Between images the filter is propagated to the controller time, so the controller always receives a prediction for the current instant.

For all variants, the heading of the pad's motion is $\mathrm{atan2}(v_y, v_x)$ of the estimated velocity, and the reported altitude is the camera height above the pad.

F. Landing state machine

The controller sends world-frame velocity commands at 50 Hz and runs the states SEARCH, TRACK, DESCEND, FINAL, CUT and LANDED, with ABORT as a recovery state.

SEARCH holds 6 m until the pad is detected. TRACK holds 4 m above the pad. The horizontal command is the estimated pad velocity (feed-forward, capped at 6 m/s)

plus a PI correction on the horizontal offset ($k_p = 1.6$, $k_i = 0.25$). The PI term is limited to 2.5 m/s, which limits the tilt needed to close the gap and so helps keep the pad in the camera's view. When the offset has been within an alignment radius of $0.15 + 0.12\,h$ m (where h is the height) and the relative speed below 0.6 m/s for 1 s, the controller enters DESCEND, which descends at 0.8 m/s above 1.5 m and 0.35 m/s below it, holds height when alignment degrades, and climbs slowly if the pad has not been seen for 0.3 s below 1 m. At 0.45 m camera height it enters FINAL and holds 0.24 m.

In FINAL, the controller predicts where the pad will be, relative to the aircraft, when the skids reach it after a motor cut:

$$t_f = \frac{v_\downarrow + \sqrt{v_\downarrow^2 + 2gh_s}}{g}, \qquad e_{\text{impact}} = e + v_{\text{rel}}\,t_f, \tag{3}$$

where h_s is the skid height above the pad, $v_\downarrow$ the current descent speed and e the horizontal offset. The motors are cut (CUT) only when $\|e_{\text{impact}}\| < 12$ cm, the relative speed is below 0.35 m/s and the latest camera fix is at most 0.15 s old. If these conditions are not met within 5 s, or the offset exceeds 0.35 m, the controller enters ABORT; it does the same if the pad is not seen for 0.7 s in DESCEND or FINAL, or if the offset in DESCEND grows well beyond the alignment radius. In ABORT the aircraft climbs back to the tracking height and retries. While the pad is out of view, the controller follows the last estimated pad position and velocity for up to 5 s.

IV. Simulation Environment

A. Aircraft, vehicle and wind

The aircraft model accepts world-frame velocity setpoints, as an autopilot's offboard interface does. A velocity loop (time constant 0.45 s with integral action) produces a desired acceleration, limited to 5 m/s^2 horizontally, which is converted into a desired roll and pitch (limited to 30°); the attitude follows with a first-order lag of 0.12 s. This keeps the coupling that matters here: the aircraft must tilt to accelerate, and the camera tilts with it [43]. Linear drag is 0.12 s^{-1}. After a motor cut the aircraft falls ballistically. Wind is an Ornstein–Uhlenbeck acceleration disturbance [44] with standard deviation 0.6 m/s^2 and time constant 2 s, with the vertical component scaled by 0.3.

The vehicle carries a 1.6 m × 1.3 m deck at 0.35 m height with the pad on top. Seven motion scenarios are used (Table I).

B. Camera rendering

The scene consists of three textured planes: a ground plane with a random-texture pattern and square clutter (which produces false marker candidates), the vehicle deck, and the pad. Each frame is rendered by warping these textures with the homography implied by the camera pose, then applying Gaussian blur ($\sigma = 0.6$ px), a random gain between 0.85 and 1.1 per run, and additive noise ($\sigma = 3$ grey levels). The camera is an ideal pinhole with 800 × 600 pixels and a 70° horizontal field of view, running at 30 Hz. In the batch runner (Section IV-D), detections become available to the estimator 50 ms after the image is taken; in the ROS 2 graph the latency is the actual processing time. Fig. 2 shows frames from one landing. Because the real detector runs on these images, effects such as perspective distortion under tilt, marker handover and markers leaving the frame near touchdown occur naturally.

TABLE I
Vehicle motion scenarios

Scenario	Motion
static	stationary
straight	2 m/s, straight line
accelerating	0.5 → 3.5 m/s over 20 s
circle	2.5 m/s on a 15 m radius
weave	2.5 m/s, sinusoidal yaw rate (0.35 rad/s, 8 s period)
stop_go	0 ↔ 3 m/s, 16 s period
fast	4.5 m/s, straight line

C. Flight-controller state errors

The estimator does not receive the true aircraft state. The reported attitude includes a slowly varying bias (0.4° standard deviation, 5 s correlation time) and white noise (0.05°); the reported velocity has 0.05 m/s white noise; and the reported position drifts as a random walk of 0.02 m/$\sqrt{\text{s}}$.

D. Execution

The same simulator, detector, estimators and controller are used in two ways: as the ROS 2 graph of Fig. 1, run in Docker, and as a single-process batch runner with the same camera rate and latency for Monte Carlo experiments. The simulation step is 5 ms. An interactive browser front end to the same code is also included; it exposes the parameters of the vehicle, wind, aircraft, camera, pad, estimator and controller and was used for exploration only, not for the results below.

V. Experiments

A. E1: pad speed from a hovering aircraft

The aircraft holds position with the camera 3.5 m above the deck while the vehicle drives underneath in a straight line, 0.6 m to one side, at a constant speed from 1 to 9 m/s, with three seeds per speed. Wind is active. All three estimators process the same detections. The number of detections per pass falls from 103 at 1 m/s to 13 at 9 m/s.

Fig. 3 and Table II show the results. On average, AC and AC+KF follow the true speed closely; CF overestimates at low speed (1.40 m/s for a 1 m/s pad) and is within 0.5 m/s of the truth from 2 m/s upward. The finite-difference variants have an RMSE of 0.48–0.98 m/s, as each difference is taken over only 33 ms. The Kalman filter reduces the RMSE to 0.07–0.14 m/s at every speed. It

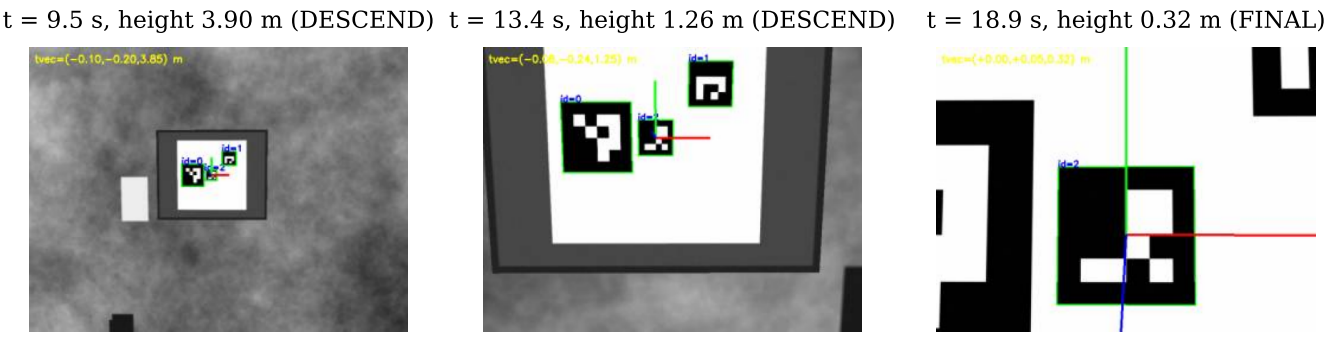


Fig. 2. Rendered downward-camera frames from the ROS 2 run on the straight scenario, with detected markers outlined and the estimated pad axes drawn. Left: all three markers visible at 3.9 m. Middle: 1.26 m. Right: 0.32 m, where only the small centre marker is fully in view.

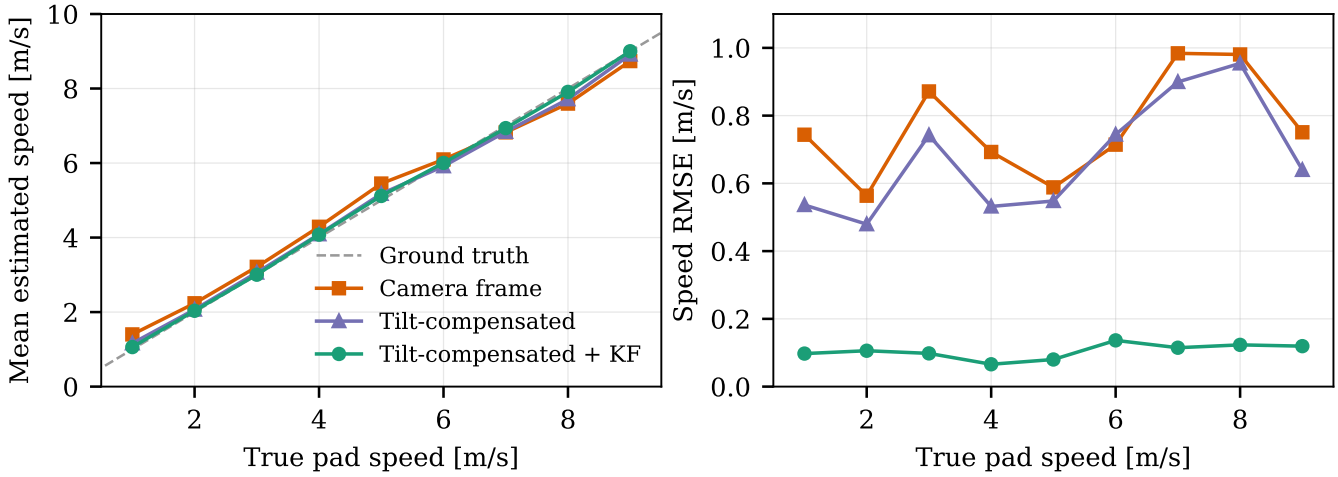


Fig. 3. E1. Left: mean estimated speed against true pad speed. Right: speed RMSE. Each point averages three passes.

TABLE II
E1 speed estimation, hovering aircraft at 3.5 m

Estimator	Mean at 1 / 9 m/s [m/s]	RMSE range [m/s]	Median heading error at 1 / 9 m/s
CF	1.40 / 8.74	0.56–0.98	31.2° / 2.0°
AC	1.16 / 8.91	0.48–0.95	10.1° / 0.9°
AC+KF	1.06 / 9.00	0.07–0.14	1.8° / 0.2°

TABLE III
E2 estimation error over 22 092 in-flight detections

Estimator	Position error [cm] median	Position error [cm] 95th pct.	Rel. speed MAE [m/s]	Altitude RMSE [cm]
CF	21.0	95.6	0.66	10.8
AC	1.4	5.5	0.15	2.5
AC+KF	2.2	6.4	0.07	1.9

produces no output for the first fixes of each pass while it initialises (or re-initialises after a gap); this accounts for 3% of samples at 1 m/s and up to 18% at the higher speeds, where a pass contains fewer detections. The median error in the direction of motion at 1 m/s is 31.2° for CF, 10.1° for AC and 1.8° for AC+KF; at 9 m/s the values are 2.0°, 0.9° and 0.2°.

B. E2: estimation accuracy in closed-loop flight

The AC+KF variant flew the aircraft in all seven scenarios with four seeds each (28 runs, up to 60 s each), while the CF and AC variants ran in parallel on the same detections. This gives 22 092 detections (21 681 with an AC+KF output), each compared with the true relative state at the image time.

Table III gives the overall errors and Fig. 4 breaks them down by tilt. The CF position error is 2.6 cm below 2° of tilt, 25.6 cm at 5°–10°, 71.5 cm at 10°–15° and 1.62 m at 20°–30°. This is the expected geometric effect: at height h, a tilt θ displaces the apparent pad position by about $h \tan\theta$. With attitude compensation the median error stays between 1.0 and 2.8 cm (AC) and between 1.7 and 3.2 cm (AC+KF) across all tilt bins. The filtered position is slightly less accurate than the raw compensated

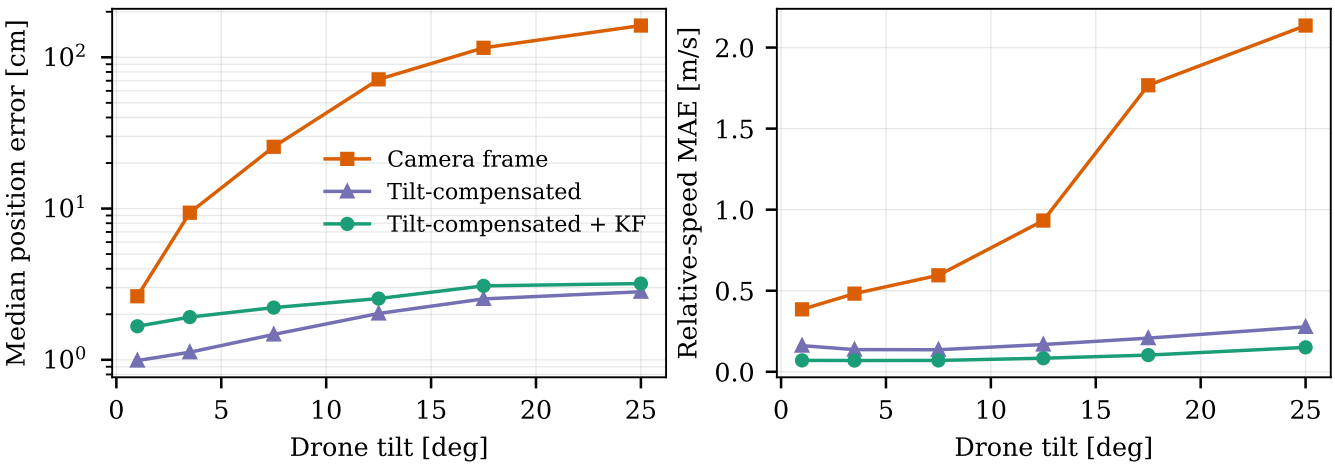


Fig. 4. E2. Median horizontal position error (left, log scale) and relative-speed mean absolute error (right) against aircraft tilt at the image time, for detections in 28 closed-loop flights. Bins: 0°–2°, 2°–5°, 5°–10°, 10°–15°, 15°–20°, 20°–30° (1485, 6136, 10 166, 3365, 681 and 259 detections for CF and AC; AC+KF has slightly fewer, as it gives no output while initialising).

TABLE IV
E3 closed-loop landings (successes out of 4 per scenario)

Estimator	stat.	str.	acc.	circ.	weave	s&g	fast	total
CF	0	0	0	0	0	0	0	0/28
AC	2	1	2	1	0	0	0	6/28
AC+KF	3	2	1	1	0	0	0	7/28

position, because the filter smooths across frames, but its relative-speed error is half as large (0.07 versus 0.15 m/s), and its median error in the pad's ground velocity is 0.05 m/s against 0.18 m/s. The CF altitude, taken directly as t_z, has an RMSE of 10.8 cm; after compensation it is 1.9 cm.

C. E3: closed-loop landing

Each estimator variant in turn drives the controller in all seven scenarios with four seeds (28 runs per variant, 84 in total). A run starts with the aircraft at 6 m height, 2.5 m horizontally from the vehicle, and ends at touchdown, after 60 s (timeout), or when the pad has not been seen for 10 s after first being tracked (lost). A run counts as a landing only if the aircraft comes to rest on the pad, within 0.35 m of the centre, with a vertical impact speed of at most 2.2 m/s and a horizontal slip of at most 0.6 m/s relative to the deck.

Table IV summarises the outcomes. All 28 CF runs ended as lost, with the pad detected in 3.4% of camera frames on average, against 74% (AC) and 54% (AC+KF). This is consistent with the tilt-dependent CF error of Fig. 4: once the aircraft tilts to follow the pad, the CF estimate places the pad tens of centimetres to metres from its true position.

With attitude compensation, landings occurred on the static, straight, accelerating and circular motions. The seven AC+KF landings ended a mean of 6.3 cm (range 2.1–14.4 cm) from the pad centre, with vertical impact speeds of 2.08–2.18 m/s and horizontal slip of 0.06–0.37 m/s. The six AC landings ended a mean of 5.3 cm (range 1.7–10.0 cm) from the centre. The mean time to touchdown for AC+KF ranged from 21.5 s (circle) to 49.0 s (accelerating).

The remaining AC+KF runs all ended in timeout (21 of 28), as did 20 of the AC runs (the other two AC failures were lost). The failed AC+KF runs went through a mean of 1.95 abort-and-retry cycles. No variant landed in the weave, stop_go or fast scenarios. The estimation errors of E2 include these failed runs, so the failures are not explained by inaccurate estimates on frames in which the pad was detected. Every abort is triggered from the DESCEND or FINAL state, that is, at low height, where the pad is detected in fewer frames (Section V-E) and the controller relies more on the filter's prediction. Identifying which abort condition dominates in each scenario is left to future work.

D. Full ROS 2 graph

To check that the node graph behaves like the batch runner, single runs were flown through the ROS 2 graph with AC+KF on four scenarios. The static run landed after 30.2 s, 11.1 cm from the centre, at 1.99 m/s vertical impact speed; the straight run landed after 19.3 s, 9.4 cm from the centre, at 2.12 m/s. The circle and weave runs timed out. Fig. 5 shows the straight run.

E. E4: pad visibility against height

Fig. 6 shows, for all 28 AC+KF runs, the fraction of camera frames in which each marker was detected, by camera height. Between 1.2 m and 4 m the pad was detected in 86–90% of frames, and all three markers contributed. Below 0.8 m the large and medium markers were detected in at most 23% of frames, and most

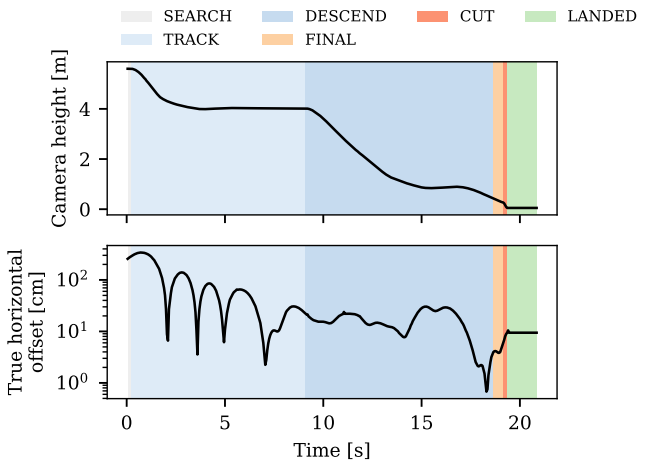


Fig. 5. ROS 2 run on the straight scenario (2 m/s) with AC+KF: camera height and true horizontal offset between aircraft and pad, with the controller state shaded.

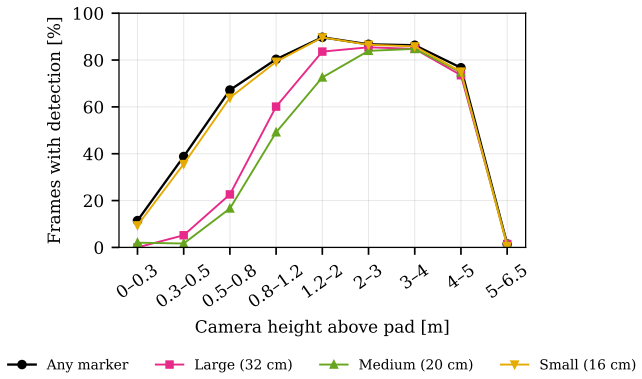


Fig. 6. E4. Fraction of camera frames with a detection of any pad marker and of each marker, against camera height, over 28 closed-loop runs.

detections came from the small centre marker. The overall detection rate fell to 67% at 0.5–0.8 m, 39% at 0.3–0.5 m and 11% below 0.3 m. Above 5 m the pad was detected in only 1.5% of frames; we did not investigate the cause of this further.

VI. Discussion

Attitude compensation. The largest single effect in these experiments is the rotation of the camera measurement by the aircraft attitude. Without it, the estimate is accurate only while the aircraft is nearly level. That is the situation in a hover test such as E1, where the CF variant's mean speed is within 0.5 m/s of the truth from 2 m/s upward, but not in closed-loop pursuit, where tilts of 5°–15° are routine (Fig. 4). In closed loop, none of the CF runs kept the pad in view. Compensation requires only the attitude that the flight controller already estimates and a timestamp match between image and attitude.

Filtering. The Kalman filter did not improve position accuracy, but it halved the relative-speed error in flight and reduced the speed RMSE by a factor of 4.5 to 10.5 relative to the unfiltered variants in the hover test. This matters for the motor-cut decision, which extrapolates the relative velocity over the fall time. The filter's prediction also supplies an estimate at the controller rate and during short gaps in detection, which are frequent below 0.5 m.

Pad design. Using markers of several sizes kept the pad detectable over the whole tracking range, as intended. The limiting factor is the final half metre, where the camera's footprint becomes smaller than the pad. The 16 cm centre marker is a compromise: a smaller marker would remain in view at lower heights but is harder to detect from further away, and a larger one leaves the frame sooner.

Landing success. With accurate estimates, closed-loop landings succeeded for steady or slowly changing vehicle motion, but the overall success rate was 25% (7 of 28) for AC+KF and no landings occurred for weaving, stop-and-go or fast motion. The mean vertical impact speed of successful landings (about 2.1 m/s) is close to the 2.2 m/s limit, because the motors are cut from a fixed hold height; the success count is likely to be sensitive to that limit and to the cut height.

Limitations. The results are from simulation only. The rendering uses flat textured planes without lighting changes, lens distortion or motion blur from rotation. The aircraft model is a velocity-controlled model rather than a rotor-level one, and ground effect and deck motion (pitch and roll from the road) are not modelled. Each closed-loop configuration was run with four seeds, so the per-scenario success counts in Table IV have wide uncertainty; in particular, the difference between 6/28 and 7/28 for AC and AC+KF is not meaningful at this sample size.

VII. Conclusion

We presented a vision-only system for landing a quadcopter on a moving ground vehicle, built from a three-marker ArUco pad, attitude-compensated pose estimation, a constant-velocity Kalman filter and a landing state machine with a predicted-impact check before the motor cut, and evaluated it in a closed-loop simulator that runs the real marker detector on rendered images. Attitude compensation reduced the median position error at 20°–30° of tilt from 1.62 m to about 3 cm, and no landing succeeded without it. In the hover test, the Kalman filter brought the speed RMSE below 0.15 m/s at all tested speeds. Landings succeeded on stationary, straight, accelerating and circling vehicles, ending within 15 cm of the pad centre, but not on weaving, stop-and-go or fast vehicles.

Future work will address the final approach for manoeuvring vehicles, where the controller currently aborts repeatedly, evaluate the pipeline on flight hardware, and replace the fixed cut height with one chosen from the predicted impact speed.

References

[1] D. Wickramanayake, R. Rajasooriya, K. Ranawella, and N. Karunarathne, "Landing a quadcopter on to a moving landing target using computer vision," Department of Computer Engineering, University of Peradeniya, 2019, https://www.researchgate.net/publication/330467111_Landing_a_Quadcopter_on_to_a_moving_landing_target_using_Computer_Vision.

[2] A. Gautam, P. B. Sujit, and S. Saripalli, "A survey of autonomous landing techniques for UAVs," in Proc. Int. Conf. Unmanned Aircraft Systems (ICUAS), 2014, pp. 1210–1218.

[3] W. Kong, D. Zhou, D. Zhang, and J. Zhang, "Vision-based autonomous landing system for unmanned aerial vehicle: A survey," in Proc. Int. Conf. Multisensor Fusion and Information Integration for Intelligent Systems (MFI), 2014, pp. 1–8.

[4] L. Xin, Z. Tang, W. Gai, and H. Liu, "Vision-based autonomous landing for the UAV: A review," Aerospace, vol. 9, no. 11, p. 634, 2022.

[5] S. Saripalli, J. F. Montgomery, and G. S. Sukhatme, "Visually guided landing of an unmanned aerial vehicle," IEEE Transactions on Robotics and Automation, vol. 19, no. 3, pp. 371–380, 2003.

[6] S. Saripalli and G. S. Sukhatme, "Landing on a moving target using an autonomous helicopter," in Field and Service Robotics, ser. Springer Tracts in Advanced Robotics. Springer, 2006, pp. 277–286.

[7] K. E. Wenzel, A. Masselli, and A. Zell, "Automatic take off, tracking and landing of a miniature UAV on a moving carrier vehicle," Journal of Intelligent & Robotic Systems, vol. 61, no. 1–4, pp. 221–238, 2011.

[8] B. Herissé, T. Hamel, R. Mahony, and F.-X. Russotto, "Landing a VTOL unmanned aerial vehicle on a moving platform using optical flow," IEEE Transactions on Robotics, vol. 28, no. 1, pp. 77–89, 2012.

[9] D. Lee, T. Ryan, and H. J. Kim, "Autonomous landing of a VTOL UAV on a moving platform using image-based visual servoing," in Proc. IEEE Int. Conf. Robotics and Automation (ICRA), 2012, pp. 971–976.

[10] F. Chaumette and S. Hutchinson, "Visual servo control. I. Basic approaches," IEEE Robotics & Automation Magazine, vol. 13, no. 4, pp. 82–90, 2006.

[11] H. Lee, S. Jung, and D. H. Shim, "Vision-based UAV landing on the moving vehicle," in Proc. Int. Conf. Unmanned Aircraft Systems (ICUAS), 2016, pp. 1–7.

[12] D. Falanga, A. Zanchettin, A. Simovic, J. Delmerico, and D. Scaramuzza, "Vision-based autonomous quadrotor landing on a moving platform," in Proc. IEEE Int. Symp. Safety, Security and Rescue Robotics (SSRR), 2017, pp. 200–207.

[13] J. Ghommam and M. Saad, "Autonomous landing of a quadrotor on a moving platform," IEEE Transactions on Aerospace and Electronic Systems, vol. 53, no. 3, pp. 1504–1519, 2017.

[14] A. Gautam, M. Singh, P. B. Sujit, and S. Saripalli, "Autonomous quadcopter landing on a moving target," Sensors, vol. 22, no. 3, p. 1116, 2022.

[15] R. E. Kalman, "A new approach to linear filtering and prediction problems," Journal of Basic Engineering, vol. 82, no. 1, pp. 35–45, 1960.

[16] A. Borowczyk, D.-T. Nguyen, A. P.-V. Nguyen, D. Q. Nguyen, D. Saussíe, and J. Le Ny, "Autonomous landing of a multirotor micro air vehicle on a high velocity ground vehicle," IFAC-PapersOnLine, vol. 50, no. 1, pp. 10 488–10 494, 2017.

[17] O. Araar, N. Aouf, and I. Vitanov, "Vision based autonomous landing of multirotor UAV on moving platform," Journal of Intelligent & Robotic Systems, vol. 85, no. 2, pp. 369–384, 2017.

[18] A. Paris, B. T. Lopez, and J. P. How, "Dynamic landing of an autonomous quadrotor on a moving platform in turbulent wind conditions," in Proc. IEEE Int. Conf. Robotics and Automation (ICRA), 2020, pp. 9577–9583.

[19] Y. Feng, C. Zhang, S. Baek, S. Rawashdeh, and A. Mohammadi, "Autonomous landing of a UAV on a moving platform using model predictive control," Drones, vol. 2, no. 4, p. 34, 2018.

[20] P. Vlantis, P. Marantos, C. P. Bechlioulis, and K. J. Kyriakopoulos, "Quadrotor landing on an inclined platform of a moving ground vehicle," in Proc. IEEE Int. Conf. Robotics and Automation (ICRA), 2015, pp. 2202–2207.

[21] K. Guo, P. Tang, H. Wang, D. Lin, and X. Cui, "Autonomous landing of a quadrotor on a moving platform via model predictive control," Aerospace, vol. 9, no. 1, p. 34, 2022.

[22] B. Hu and S. Mishra, "Time-optimal trajectory generation for landing a quadrotor onto a moving platform," IEEE/ASME Transactions on Mechatronics, vol. 24, no. 2, pp. 585–596, 2019.

[23] A. Paris, A. Tagliabue, and J. P. How, "Autonomous MAV landing on a moving platform with estimation of unknown turbulent wind conditions," in AIAA Scitech 2021 Forum, 2021.

[24] T. Baca, P. Stepan, V. Spurny, D. Hert, R. Penicka, M. Saska, J. Thomas, G. Loianno, and V. Kumar, "Autonomous landing on a moving vehicle with an unmanned aerial vehicle," Journal of Field Robotics, vol. 36, no. 5, pp. 874–891, 2019.

[25] M. Beul, M. Nieuwenhuisen, J. Quenzel, R. A. Rosu, J. Horn, D. Pavlichenko, S. Houben, and S. Behnke, "Team NimbRo at MBZIRC 2017: Fast landing on a moving target and treasure hunting with a team of micro aerial vehicles," Journal of Field Robotics, vol. 36, no. 1, pp. 204–229, 2019.

[26] J. M. Daly, Y. Ma, and S. L. Waslander, "Coordinated landing of a quadrotor on a skid-steered ground vehicle in the presence of time delays," Autonomous Robots, vol. 38, no. 2, pp. 179–191, 2015.

[27] R. Polvara, M. Patacchiola, S. Sharma, J. Wan, A. Manning, R. Sutton, and A. Cangelosi, "Toward end-to-end control for UAV autonomous landing via deep reinforcement learning," in Proc. Int. Conf. Unmanned Aircraft Systems (ICUAS), 2018, pp. 115–123.

[28] A. Rodriguez-Ramos, C. Sampedro, H. Bavle, P. de la Puente, and P. Campoy, "A deep reinforcement learning strategy for UAV autonomous landing on a moving platform," Journal of Intelligent & Robotic Systems, vol. 93, no. 1–2, pp. 351–366, 2019.

[29] N. Koenig and A. Howard, "Design and use paradigms for Gazebo, an open-source multi-robot simulator," in Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS), vol. 3, 2004, pp. 2149–2154.

[30] P. Ladosz, M. Mammadov, H. Shin, W. Shin, and H. Oh, "Autonomous landing on a moving platform using vision-based deep reinforcement learning," IEEE Robotics and Automation Letters, vol. 9, no. 5, pp. 4575–4582, 2024.

[31] E. Olson, "AprilTag: A robust and flexible visual fiducial system," in Proc. IEEE Int. Conf. Robotics and Automation (ICRA), 2011, pp. 3400–3407.

[32] J. Wang and E. Olson, "AprilTag 2: Efficient and robust fiducial detection," in Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS), 2016, pp. 4193–4198.

[33] S. Garrido-Jurado, R. Muñoz-Salinas, F. J. Madrid-Cuevas, and M. J. Marín-Jiménez, "Automatic generation and detection of highly reliable fiducial markers under occlusion," Pattern Recognition, vol. 47, no. 6, pp. 2280–2292, 2014.

[34] F. J. Romero-Ramirez, R. Muñoz-Salinas, and R. Medina-Carnicer, "Speeded up detection of squared fiducial markers," Image and Vision Computing, vol. 76, pp. 38–47, 2018.

[35] T. Collins and A. Bartoli, "Infinitesimal plane-based pose estimation," International Journal of Computer Vision, vol. 109, no. 3, pp. 252–286, 2014.

[36] F. J. Romero-Ramirez, R. Muñoz-Salinas, and R. Medina-Carnicer, "Fractal markers: A new approach for long-range marker pose estimation under occlusion," IEEE Access, vol. 7, pp. 169 908–169 919, 2019.

[37] A. Khazetdinov, A. Zakiev, T. Tsoy, M. Svinin, and E. Magid, "Embedded ArUco: a novel approach for high precision UAV landing," in Proc. Int. Siberian Conf. Control and Communications (SIBCON), 2021, pp. 1–6.

[38] J. Wubben, F. Fabra, C. T. Calafate, T. Krzeszowski, J. M. Marquez-Barja, J.-C. Cano, and P. Manzoni, "Accurate landing of unmanned aerial vehicles using ground pattern recognition," Electronics, vol. 8, no. 12, p. 1532, 2019.

[39] I. Lebedev, A. Erashov, and A. Shabanova, "Accurate autonomous UAV landing using vision-based detection of ArUco-

marker," in Interactive Collaborative Robotics (ICR 2020), ser. Lecture Notes in Computer Science. Springer, 2020, pp. 179–188.
[40] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot operating system 2: Design, architecture, and uses in the wild," Science Robotics, vol. 7, no. 66, p. eabm6074, 2022.
[41] G. Bradski, "The OpenCV library," Dr. Dobb's Journal of Software Tools, 2000.
[42] L. Meier, D. Honegger, and M. Pollefeys, "PX4: A node-based multithreaded open source robotics framework for deeply embedded platforms," in Proc. IEEE Int. Conf. Robotics and Automation (ICRA), 2015, pp. 6235–6240.
[43] R. Mahony, V. Kumar, and P. Corke, "Multirotor aerial vehicles: Modeling, estimation, and control of quadrotor," IEEE Robotics & Automation Magazine, vol. 19, no. 3, pp. 20–32, 2012.
[44] G. E. Uhlenbeck and L. S. Ornstein, "On the theory of the brownian motion," Physical Review, vol. 36, no. 5, pp. 823–841, 1930.